

N8N Automations

The Brief

59 business chores, each built as a flowchart that then runs itself

1 The project in one page

Fifty-nine business processes, each built as a flowchart inside a tool called n8n, exported to text files, and documented one folder at a time. Scraping leads, chasing enquiries, sending invoices, writing posts, rendering videos, answering questions from a company's own documents.

There is no program here to compile and no test suite to run. The unit of work is a *workflow*: a chain of steps wired together on a canvas, started by an event, that moves data between web services and, more often than not, hands something to an AI model along the way.

The three numbers that describe this repository

- 59** automation folders, each with a README, a credential manifest, a licence and an export
- 80** exported workflow files (six folders hold multi-workflow systems)
- 1,645** nodes, of which 186 are sticky notes documenting the other 1,459

All three were verified by parsing the files, not by counting the README's bullets. The headline claim of **59 automations is accurate**.

2 What n8n is, for a reader who has never seen it

Take a real chore: *when a deal is marked won in our customer database, create a customer in our payment processor and invoice them*. You can do it by hand in four minutes, or write and host a program for a day. n8n is the third option: drag a box for each step, wire the boxes together, and it runs the flowchart for you. It already knows how to talk to hundreds of services.

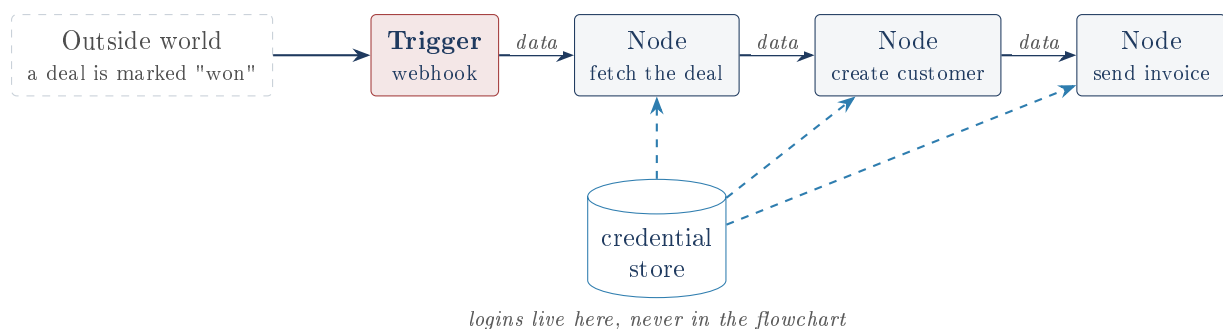


Figure 1: The whole vocabulary. Each box is a **node**. Each arrow is a **connection**. The box that starts it is the **trigger**. The logins are **credentials**, referenced from a separate store. The chain is a **workflow**.

Dragging boxes removes the typing, not the thinking

Everything hard about software survives the move to a canvas: state, failure, retries, duplicate events, data shape mismatches, rate limits. Section 7 is a list of places in this repository where

that bill came due, and every item on it is an ordinary software bug that would have been equally possible in handwritten code.

3 What the collection is made of

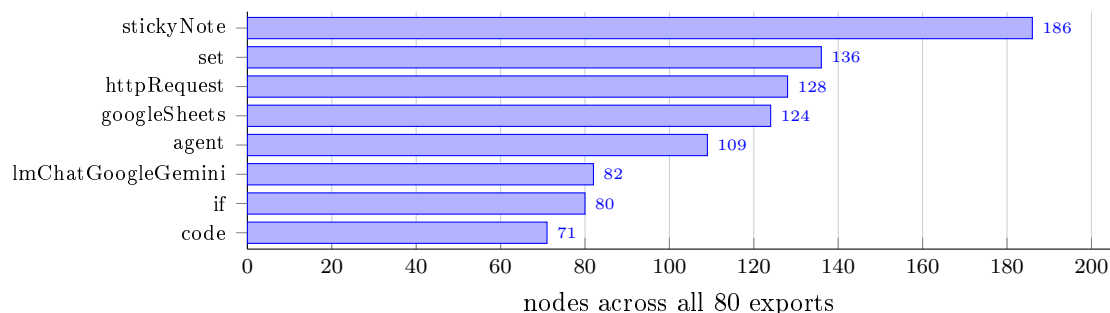


Figure 2: The eight most common node types, and a fair portrait of the whole collection: moving data between web services and an LLM, glued together with spreadsheets, and unusually well commented.

Four readings fall straight out of that chart, and each is worth a sentence.

Sticky notes are the most common node. A sticky note does nothing at run time; it is a comment on the canvas. One node in nine here exists only to explain the others. That is a genuinely unusual signal of care.

Google Sheets is the database (124 nodes). It is the lead list, the job queue, the state store, the log, and the client’s review screen, all at once.

128 hand-written API calls. Wherever n8n had no purpose-built node, the author read the vendor’s docs and constructed the raw call. This is the single most important thing to notice about the collection: it is where the drag-and-drop tool runs out and the engineering starts.

AI is structural, not decorative. 109 agent nodes, 82 Gemini models, 37 OpenAI nodes. About a fifth of the repository.

4 The seven families

Fifty-nine is too many to list, and they are not 59 unrelated things. Grouping them by what they do (this grouping is this document’s own reading; the repository does not group its folders) gives seven families whose architecture is largely predictable from the family alone.

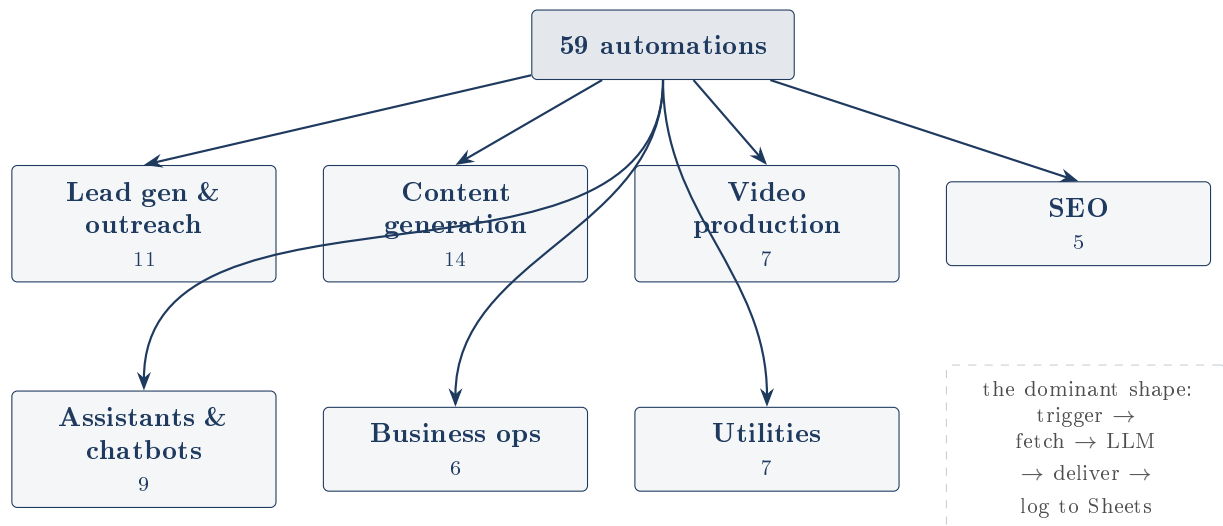


Figure 3: Counts are of folders and sum to 59.

The extremes are worth knowing:

- **Smallest automation:** 3 nodes, youtube-transcript-fetcher (a webhook, an API call, a reply).
- **Largest single file:** 190 nodes, social-media-content-generator.
- **Largest system:** 10 files and 275 nodes, cold-email-outreach-sequence-system.

5 Three automations worth understanding

5.1 The one that bills people

crm-lead-to-stripe-invoice is 6 nodes and the best single example in the repository, because it is small enough to hold in your head and its flaws cost real money.

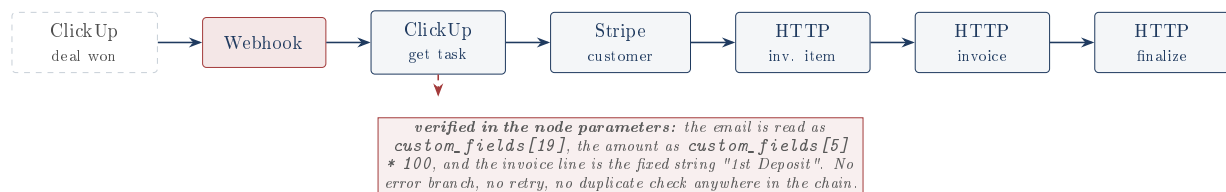


Figure 4: Six nodes, one straight line, no error path.

Note the shape: a native Stripe node creates the customer, then three hand-written HTTP calls do the invoicing, because n8n's Stripe node does not implement Stripe's three-step invoice dance. Native node where it fits, raw API where it does not, same stored credential reused for both. That move is the defining habit of the whole collection.

Two real defects, both verified against the file

Fields are read by array position. `custom_fields[19]` is the email *today*. Reorder a column in ClickUp and it silently becomes something else. The workflow does not fail; it invoices the wrong data to the wrong address, and the first person to notice is the customer. **No idempotency, on a node that charges people.** No check for an existing customer, no check for an existing invoice. Webhook sends retry as a matter of routine. Two deliveries produce two invoices. This is the highest-severity issue in the repository and the only one

that costs money on its first occurrence.

5.2 The one with the best architecture

ultimate-personal-assistant-system: five files, 50 nodes. You text a Telegram bot, by typing or by voice. An orchestrating agent works out what you want and delegates to one of four specialist agents.

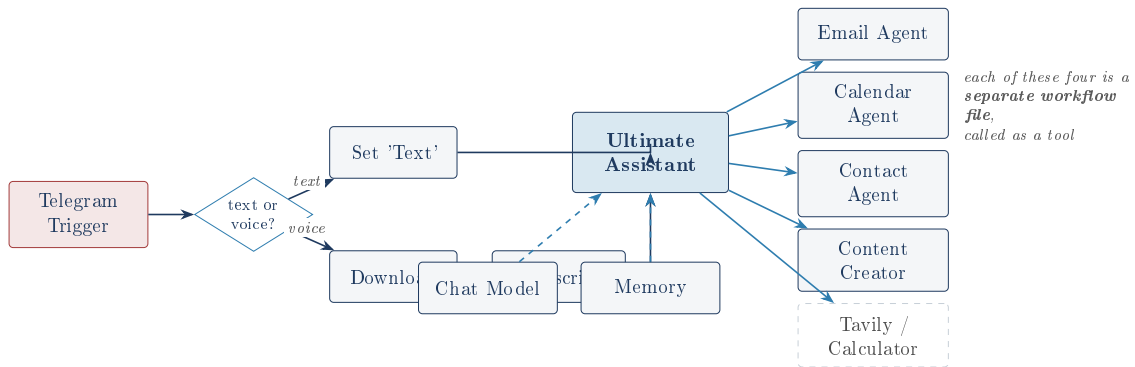


Figure 5: Solid blue arrows out of the agent are tool calls the model chooses to make. Dashed arrows are equipment attached to it.

Sub-workflow as tool is the automation equivalent of a function call

Each specialist is its own file with its own entry point, which buys what functions buy in code: each can be tested alone, a change to contacts cannot break email, and each agent carries a small focused prompt instead of one enormous one. The voice branch converges back before the agent, so the agent never knows which input path it came from. That is the right place for that seam.

The contrast is instructive: **social-media-content-generator** solves a comparable problem on a single 190-node canvas.

5.3 The one with the most interesting internals

rag-ingestion-pipeline, 47 nodes. It watches a Google Drive folder, and when a document appears it works out the type, extracts the text, chops it up, converts the chunks to embeddings and stores them in a Supabase vector table. A separate chat agent answers questions from that table.

Jargon: RAG (Retrieval-Augmented Generation)

A way to make an AI model answer from *your* documents rather than its general training. Chop the documents into chunks, convert each into a list of numbers capturing its meaning (an *embedding*), and store them. At question time, embed the question, find the nearest chunks, and paste them into the prompt. The model then answers from real text you supplied.

Two details show genuine understanding. Spreadsheets are summarized into prose *before* embedding, because embedding a spreadsheet row by row produces chunks like "Q3 | 41 | 12" that carry no retrievable meaning. And on the update path, old rows are deleted before new ones are inserted, because otherwise an edited document leaves its old chunks behind and the agent answers from a version that no longer exists.

Half of it is a copy of the other half, and that half is switched off

The file contains two complete, near-identical ingestion pipelines: one driven by a `File Created` trigger, one by a `File Updated` trigger that is **disabled**. Same nodes, different names (`Switch` and `Switch2`, `Aggregate` and `Aggregate1`).

Every future change to the extraction logic must be made twice, in two places, with confusingly similar names, and a mistake in the dead copy stays invisible until someone re-enables it. This is copy-paste architecture, and it is the characteristic debt of visual tools: the canvas makes duplicating a branch trivially easy and offers nothing that would encourage factoring it out.

6 The habits worth defending

6.1 Google Sheets as the database

Defensible, with a known ceiling

The instinct is to call it amateurish. A spreadsheet buys four things a real database does not: the client can see it, the client can edit it (human review, for free), it needs no provisioning, and it was already the incumbent tool. For a small agency automating its own chores, that is an engineering decision, not a shortcut.

It breaks on **concurrency** (no transactions, so two runs can both read "not yet emailed" and both email), on **uniqueness** (no constraint, which is precisely why the cold email system can push a lead twice), and on **scale** (one SEO workflow reads its entire post history on every run). Right call at this size; the ceiling is real and stated.

6.2 Trusting the AI's output shape

The most common bug class here. A prompt says "return only JSON", and nothing downstream checks that it did. `n8n` has a structured output parser node, which forces the response into a declared schema and fails loudly when it does not fit. It appears **3 times in 1,645 nodes**, in just 2 of the 80 files. In the other 78, output shape is a matter of hoping.

The verified worst case, and the lesson in it

In `competitor-content-gap-analyzer`, the `Clean Content` node's stored JavaScript has had its newlines collapsed into literal letter `n` characters, so the code reads `{nnnn$input.item.json.content = ...}`. That throws a `ReferenceError` the instant it runs. Its regular expressions are also passed as quoted strings rather than regex literals, so they would match nothing even if the syntax were fixed. The node has never cleaned anything.

The part that matters is what happens next: the node is set to `continueOnFail: true`, so the error is swallowed, the item passes through untouched, and **the workflow reports success**. Every run has produced a content gap analysis based on no content.

The lesson is not that someone mangled a code node. It is that an error-tolerance flag turned a loud crash into a silent wrong answer. That flag is set on 27 nodes here.

7 The honest assessment

7.1 Genuinely good

- **Documentation discipline is exceptional:** 59 READMEs on nine consistent headings, 59 credential manifests, 9,304 lines of prose, plus 186 sticky notes inside the canvases.

- **The self-criticism is real and specific.** The READMEs say "the content-cleaning code is broken" and "custom fields are read by array index". Spot-checks against the raw JSON confirmed these claims exactly, including the subtle ones. This is rarer than it should be.
- **No credentials leaked.** See below.
- **The hard parts were not dodged:** 128 hand-written API calls, async polling, a considered RAG chunking strategy, a properly factored sub-workflow architecture where it is used.

7.2 Weak, ordered by what it would cost

1. **No idempotency on the invoicing workflow.** A retried webhook double-bills a real customer.
2. **Silent failures via `continueOnFail`** (27 nodes). Proven, not theoretical.
3. **Copy-paste architecture:** the duplicated RAG branch; six duplicated copywriting blocks in the cold email system; nine orphaned call nodes left from an old version.
4. **Orphaned nodes in 12 of the 80 files**, 55-plus nodes with no path from any trigger. On a canvas they look exactly like working steps.
5. **Unbounded polling loops.** No attempt cap anywhere in the video family.
6. **Hardcoded values where expressions belong:** a fixed \$1,850 cost, "Hey Nick" in a client email, the literal caption "hello" on every post in one video workflow.
7. **Documentation drift:** sticky notes labelling reminder stages "5-Minute Alert" when the node conditions say one hour.

7.3 Security

No live credential is committed, and this was checked properly

n8n stores logins in an encrypted credential store; a node holds only a pointer to one. Verified across all 80 files: **373 credential references**, 27 distinct credential types, every one containing exactly `id` and `name`. Not one carries a value.

That is n8n's design doing the work. What the author was responsible for is everything n8n does not protect, and that was audited too: all provider-key formats scanned across all 268 tracked files (three hits, all placeholders such as `Bearer YOUR_API_KEY_HERE`); every referenced host is a public vendor API, with no instance hostname and no Supabase project URL; every email is a placeholder; the `shared` block that normally carries the account owner's name and email is scrubbed in all 30 files that have one.

The project README says live keys were found and stripped. This audit found no survivor.

The scrub was manual, and that is the residual risk

One field, the n8n account name, appears with **sixteen different placeholder spellings** across the files (`YOUR_EMAIL_HERE`, `owner@example.com`, `Portfolio Owner`, `REDACTED_USER`, and twelve more). The outcome is correct, but sixteen spellings of one substitution is the signature of a hand pass, and a hand pass is one that can miss a file. A scripted scrub with a single canonical placeholder would be safer, because it could be re-run to *prove* nothing was missed. Today the proof is an audit, which is only a snapshot.

7.4 The maturity question

The one thing to get straight before any interview

28 manual triggers (the most common trigger type here). **9 of 80** workflows marked active. **45** disabled nodes. **12** files with orphans.

A manual trigger is what n8n gives you while you are *building*, so you can press Execute and watch data flow. A deployed automation is normally started by a schedule, a webhook or an event.

The defensible claim is: *59 automations designed and built across seven problem domains, every one documented including its faults, with a clear account of which are production-hardened and which are at demonstration maturity.* That is a strong claim and it survives someone opening the files.

The claim to avoid is "59 automations run my client's business". The repository does not contain the evidence for it: execution logs are not here, and the active flags say what they say.

7.5 What is inferred rather than confirmed

The seven-family taxonomy is this document's own reading; the repository does not group its folders, and while the folder counts are exact the boundaries are a judgment call. The claim that the collection is "built and demonstrated rather than deployed" is a strong inference from trigger types and active flags, but the alternative (that workflows were routinely deactivated before export) cannot be ruled out from the files alone. In the cold email system, the handoff from the research workflow to the orchestrator is inferred from matching field names; there is no connection in the JSON, and the project's own README is equally explicit about that. Whether the disabled trigger in the RAG pipeline was switched off deliberately or abandoned mid-refactor cannot be determined from the export. And whether the assistants behave *well* is untestable from the files: the graphs are sound, but the behaviour lives in prompts, and prompts cannot be evaluated by reading them.

All counts were produced by parsing the 80 exported workflow files directly. The full treatment, including the data model of an export, walkthroughs of the largest systems, and the complete audit, is in the companion Deep Dive.